Programs 1A to 7

Program 1A

```
from tensorflow.keras.datasets import cifar10
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train[:5000]
y_train = y_train[:5000]

x_train = x_train.reshape(5000, 3072)
x_test = x_test.reshape(10000, 3072)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train.ravel())

y_pred = knn.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)

print("KNN Accuracy:", round(accuracy, 2))
```

Program 1B

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = Sequential()
model.add(Flatten(input_shape=(32, 32, 3)))
model.add(Dense(256, activation='relu'))
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
```

```
        metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=64)

loss, accuracy = model.evaluate(x_test, y_test)

print("3-Layer Neural Network Accuracy:", round(accuracy * 100, 2), "%")
```

Program 2

```
import numpy as np
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input,
decode_predictions
from tensorflow.keras.preprocessing import image

model = ResNet50(weights='imagenet')

img_path = "cat.jpg"
img = image.load_img(img_path, target_size=(224, 224))

img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

predictions = model.predict(img_array)
result = decode_predictions(predictions, top=1)[0]

print("Prediction:")
print(result)
```

Program 3

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
```

```
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=64)

loss, accuracy = model.evaluate(x_test, y_test)

print("CNN Accuracy:", round(accuracy * 100, 2), "%")
```

Program 4

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, BatchNormalization, MaxPooling2D, Dropout,
Flatten, Dense

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))
model.add(Dropout(0.25))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])
```

```
model.fit(x_train, y_train, epochs=5, batch_size=64)

loss, accuracy = model.evaluate(x_test, y_test)

print("Tuned CNN Accuracy:", round(accuracy * 100, 2), "%")
```

Program 5

```
import torch
import torchvision
from PIL import Image
from torchvision import transforms

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()

img = Image.open("object.jpg").convert("RGB")

transform = transforms.ToTensor()
img_tensor = transform(img)

with torch.no_grad():
  prediction = model([img_tensor])

print("Objects Detected:")

for i in range(len(prediction[0]["scores"])):
  score = prediction[0]["scores"][i].item()
  if score > 0.8:
    label = prediction[0]["labels"][i].item()
    print("Object", i + 1, "Label:", label, "Confidence:", round(score, 2))
```

Program 6

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

max_features = 5000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
```

```
x_train = pad_sequences(x_train, maxlen=500)
x_test = pad_sequences(x_test, maxlen=500)

model = Sequential()
model.add(Embedding(max_features, 32))
model.add(SimpleRNN(100))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=1, batch_size=64)

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", round(accuracy, 4))
```

Program 7

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

training_data = [
   "hello hi",
   "how are you fine",
   "what is your name chatbot",
   "bye goodbye"
]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(training_data)

total_words = len(tokenizer.word_index) + 1

input_sequences = []
for line in training_data:
   token_list = tokenizer.texts_to_sequences([line])[0]
   input_sequences.append(token_list)
```

```
max_len = max(len(seq) for seq in input_sequences)

input_sequences = pad_sequences(
   input_sequences,
   maxlen=max_len,
   padding='post'
)

X = input_sequences[:, :-1]
y = input_sequences[:, -1]

y = to_categorical(y, num_classes=total_words)

model = Sequential()
model.add(Embedding(total_words, 64, input_length=max_len - 1))
model.add(Bidirectional(LSTM(20)))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='categorical_crossentropy',
            optimizer='adam',
            metrics=['accuracy'])

model.fit(X, y, epochs=200, verbose=0)

print("Chatbot Ready!")

while True:
   user = input("You : ")

   if user.lower() == "exit":
      print("Chatbot : Goodbye")
      break
   elif "hello" in user.lower():
      print("Chatbot : Hi")
   elif "how are you" in user.lower():
      print("Chatbot : I am fine")
   elif "name" in user.lower():
      print("Chatbot : I am a chatbot")
   elif "bye" in user.lower():
      print("Chatbot : Goodbye")
   else:
      print("Chatbot : Sorry, I don't understand")
```